

Shortest Path Algorithms

Names and ids:

Name1: البراء مصطفى محمد

ID1: 22010636

Name2: فاروق اشرف فاروق

ID2: 22011012

Name3: عبد الرحمن سعيد رمضان جمعه

ID3: 22010879

Name4: محمد عمرو محمد سعيد محمود الرمال

ID4: 22011153

Name5: ساهر طارق أنور زياد

ID5: 19015763

Introduction

This report presents a comprehensive analysis of three fundamental shortest path algorithms implemented in the Graph class provided:

1. **Dijkstra's Algorithm** - For graphs with non-negative edge weights
2. **Bellman-Ford Algorithm** - For graphs that may contain negative edge weights
3. **Floyd-Warshall Algorithm** - For finding shortest paths between all pairs of vertices

Algorithm Overview

Dijkstra's Algorithm

Dijkstra's algorithm solves the single-source shortest path problem for graphs with non-negative edge weights. The implementation in the Graph class uses a priority queue to efficiently select the next vertex with the minimum distance, which is a common optimization.

Key features:

- Cannot handle negative edge weights
- Use a greedy approach to find optimal paths
- Processes vertices in order of increasing distance from the source

Bellman-Ford Algorithm

The Bellman-Ford algorithm also solves the single-source shortest path problem but can handle graphs with negative edge weights. It can also detect negative cycles, which are cycles in the graph where the total edge weight is negative.

Key features:

- Can handle negative edge weights
- Can detect negative cycles
- Processes all edges in each iteration

Floyd-Warshall Algorithm

The Floyd-Warshall algorithm solves the all-pairs shortest path problem, finding the shortest paths between every pair of vertices in the graph.

Key features:

- Computes the shortest paths between all pairs of vertices
- Can handle negative edge weights
- Can detect negative cycles
- Uses a dynamic programming approach

Time Complexity Analysis

Dijkstra's Algorithm

The time complexity of Dijkstra's algorithm depends on the data structure used for the priority queue:

- **Operations:** Each vertex is added to and removed from the priority queue once
- **Priority Queue Operations:** The implementation uses a binary heap (Priority Queue in Java)
- **Edge Relaxation:** Each edge is examined at most once

Time complexity analysis:

- Vertex extraction: $O(|V| \log |V|)$ - Each extraction takes $O(\log |V|)$ time, done $|V|$ times
- Edge relaxation: $O(|E| \log |V|)$ - Each relaxation involves a priority queue update which takes $O(\log |V|)$ time

Overall time complexity: $O((|V| + |E|) \log |V|)$

In the provided implementation, the use of a PriorityQueue with Edge objects results in this optimal complexity.

Bellman-Ford Algorithm

Bellman-Ford performs a fixed number of iterations over all edges:

- **Main Loop:** Iterates $|V| - 1$ times (where $|V|$ is the number of vertices)
- **Inner Loop:** Examines all $|E|$ edges in each iteration
- **Negative Cycle Detection:** One additional pass through all edges

Time complexity analysis:

- Main processing: $O(|V| \times |E|)$
- Negative cycle detection: $O(|E|)$

Overall time complexity: $O(|V| \times |E|)$

This makes Bellman-Ford slower than Dijkstra's algorithm for graphs without negative weights, but it's more versatile.

Floyd-Warshall Algorithm

Floyd-Warshall uses a dynamic programming approach with three nested loops:

- **Outer Loop:** Iterates through all $|V|$ vertices as intermediate vertices
- **Middle Loop:** Iterates through all $|V|$ vertices as source vertices
- **Inner Loop:** Iterates through all $|V|$ vertices as destination vertices

Time complexity analysis:

- **Overall time complexity:** $O(|V|^3)$

This cubic time complexity makes Floyd-Warshall suitable only for small to medium-sized graphs, but it computes all the shortest paths in a single execution.

Space Complexity Analysis

Dijkstra's Algorithm

The space requirements for Dijkstra's algorithm in the implementation are:

- **Distance Array:** $O(|V|)$ - Stores the distance from the source to each vertex
- **Parent Array:** $O(|V|)$ - Stores the parent of each vertex in the shortest path tree
- **Visited Array:** $O(|V|)$ - Tracks visited vertices
- **Priority Queue:** $O(|V|)$ - At most, all vertices could be in the queue
- **Adjacency List:** $O(|V| + |E|)$ - Graph representation

Overall space complexity: $O(|V| + |E|)$

Bellman-Ford Algorithm

Bellman-Ford's space requirements are simpler:

- **Distance Array:** $O(|V|)$ - Stores the distance from the source to each vertex
- **Parent Array:** $O(|V|)$ - Stores the parent of each vertex in the shortest path tree
- **Edge List:** $O(|E|)$ - Graph representation

Overall space complexity: $O(|V| + |E|)$

Floyd-Warshall Algorithm

Floyd-Warshall requires matrices to store distances and predecessors:

- **Costs Matrix:** $O(|V|^2)$ - Stores distances between all pairs of vertices
- **Predecessor Matrix:** $O(|V|^2)$ - Stores the predecessor of each vertex in the shortest path

Overall space complexity: $O(|V|^2)$

This quadratic space requirement can be prohibitive for large graphs.

Performance comparison

1. Single-Source Shortest Path

Graph Characteristics	Dijkstra	Bellman-Ford	Floyd-Warshall
Sparse Graph ($ E \approx V $)	$O(V + E \log V)$	$O(V E)$	$O(V ^3)$
Dense Graph ($ E \approx V ^2$)	$O(V ^2 \log V)$	$O(V ^3)$	$O(V ^3)$
Negative Weights	Not applicable	$O(V E)$	$O(V ^3)$

Interpretation:

1. Sparse Graphs ($|E| \approx |V|$):

- **Dijkstra** is optimal for non-negative weights.
- **Bellman-Ford** is slower ($O(|V|^2)$) but essential for negative weights or detecting negative cycles.
- **Floyd-Warshall** is overkill for SSSP due to its all-pairs design ($O(|V|^3)$).

2. Dense Graphs ($|E| \approx |V|^2$):

- **Dijkstra's** runtime degrades to $O(|V|^2 \log |V|)$, narrowing the gap with **Bellman-Ford** ($O(|V|^3)$).
- **Floyd-Warshall** remains impractical for SSSP, even in dense graphs.

3. Negative Weights:

- **Dijkstra** fails with negative weights.
 - **Bellman-Ford** is the best for SSSP with negative weights.
 - **Floyd-Warshall** can solve SSSP but is inefficient unless all-pairs paths are needed.
-

2. All-Pairs Shortest Path

Graph Characteristics	Dijkstra	Bellman-Ford	Floyd-Warshall
Sparse Graph ($ E \approx V $)	$O(V ^2 \log V)$	$O(V ^3)$	$O(V ^3)$
Dense Graph ($ E \approx V ^2$)	$O(V ^3 \log V)$	$O(V ^4)$	$O(V ^3)$
Negative Weights	Not applicable	$O(V ^2 E) \approx O(V ^4)$	$O(V ^3)$

Interpretation:

1. Sparse Graphs:

- **Dijkstra** is efficient ($O(|V|^2 \log |V|)$), outperforming **Floyd-Warshall** ($O(|V|^3)$).
- **Bellman-Ford** is less efficient ($O(|V|^3)$).

2. Dense Graphs:

- **Floyd-Warshall** becomes the best choice ($O(|V|^3)$), beating **Dijkstra** ($O(|V|^3 \log |V|)$).
- **Bellman-Ford** is slow ($O(|V|^4)$).

3. Negative Weights:

- **Floyd-Warshall** is the most efficient ($O(|V|^3)$) and handles negative weights.
- **Bellman-Ford** is impractical ($O(|V|^4)$).

-
- **SSSP:** Prefer **Dijkstra** for non-negative weights; **Bellman-Ford** for negative weights.
 - **APSP:** Use **Dijkstra** for sparse graphs; **Floyd-Warshall** for dense graphs or negative weights.
 - **Negative Cycles:** **Bellman-Ford** (SSSP) or **Floyd-Warshall** (APSP) can detect them.

Practical Considerations

Mean Time to Find Shortest Path Between Two Specific Nodes

1. **Small Graphs** ($|V| < 1000$):
 - Dijkstra's algorithm is generally the fastest solution for single-pair queries when weights are non-negative
 - If the graph is very dense or has negative weights, pre-computing all shortest paths with Floyd-Warshall may be more efficient
2. **Medium Graphs** ($1000 \leq |V| < 10000$):
 - Dijkstra's algorithm remains efficient for on-demand path queries
 - Pre-computing with Floyd-Warshall becomes memory-intensive
 - Bellman-Ford should only be used if negative weights are present
3. **Large Graphs** ($|V| \geq 10000$):
 - Floyd-Warshall becomes impractical due to $O(|V|^2)$ memory requirements
 - Dijkstra's algorithm is the preferred solution for most scenarios
 - For graphs with negative weights, Bellman-Ford may be the only option.

Time to Find Shortest Paths Between All Pairs of Nodes

1. **Small Graphs** ($|V| < 100$):
 - Floyd-Warshall is the most straightforward and often the most efficient solution
 - The $O(|V|^3)$ time complexity is acceptable for small graphs
2. **Medium Graphs** ($100 \leq |V| < 1000$):
 - For sparse graphs, running Dijkstra from each source may be faster than Floyd-Warshall
 - For dense graphs, Floyd-Warshall is generally more efficient
3. **Large Graphs** ($|V| \geq 1000$):
 - Both approaches become computationally expensive

Conclusion

Based on the analysis of the three shortest path algorithms:

1. **Dijkstra's Algorithm** is the most efficient choice for single-source shortest path problems in graphs with non-negative edge weights. It performs particularly well in sparse graphs.
2. **Bellman-Ford Algorithm** is necessary when the graph may contain negative edge weights. While slower than Dijkstra's algorithm, it provides the important capability to detect negative cycles.
3. **Floyd-Warshall Algorithm** is the most suitable for all-pairs shortest path problems, especially in dense graphs or when paths need to be queried frequently.

When selecting an algorithm for a specific application:

- number of vertices and edges
- Evaluate whether negative edge weights are present
- Determine if single-source or all-pairs paths are needed
- The frequency of path queries

GitHub Repo

<https://github.com/farouk36/Shortest-Paths-Algorithms>